

[5] Analytical Engines with Context-Rich Processing: Towards Efficient Next-Generation Analytics 총정리

저자: Viktor Sanca, Anastasia Ailamaki (EPFL — Data-Intensive Applications and Systems Lab)

학회/년도: ICDE 2023 (IEEE 40th International Conference on Data Engineering) — Vision Paper

분량: 약 8페이지

역할군: (D) 이론적 기반; 임베딩을 관계형 대수에 통합하는 비전

요약

이 논문은 [6]번 논문(Context-Enhanced Relational Operators with Vector Embeddings)의 전신이 되는 비전 논문으로, 임베딩을 관계형 대수에 통합하는 아이디어의 출발점이다. 비전 논문(Vision Paper)이란 구체적인 실험보다는 새로운 연구 방향을 제시하고 커뮤니티의 논의를 유도하는 논문으로, [6]의 기술적 기여를 이해하기 위한 철학적·개념적 배경이 된다.

핵심 메시지는 현대 데이터의 80~90%가 비구조화 데이터인데, 관계형 DBMS는 여전히 구조화 테이블에 최적화되어 있다는 것이다. 따라서 임베딩 연산자(E-Operator)를 관계형 대수의 공식 연산자로 정의하면, 기존 최적화 규칙(선택 푸시다운, 조인 재정렬 등)을 임베딩 연산에도 자동으로 적용할 수 있다. 이것이 바로 VAQ(Vector-Augmented Analytical Query) 최적화의 이론적 토대가 된다.

본 논문과의 관계: 비전 논문으로서 구체적인 실험은 없지만, "임베딩을 관계형 대수에 통합하면 어떤 이득을 얻을 수 있을까"라는 근본적 질문에 대한 답을 제시하며, Exqutor가 이를 실행하는 데 필요한 정확한 카디널리티 추정의 필요성을 이론적으로 정당화한다.

상세분석

5.1 해결하고자 하는 문제: 데이터의 본질적 변화

데이터 생태계의 변화

과거(2010년대 초):

데이터 구성:

- 구조화: 80% (숫자, 날짜, 범주형)
- 비구조화: 20% (이미지, 텍스트)

도구 분포:

- RDBMS 투자: 80% (SQL, 트랜잭션, ACID)
- 비구조화 처리: 20% (특수 시스템)

현대(2020년대):

- 구조화: 10~20% (메타데이터만 해당)
- 비구조화: 80~90% (텍스트, 이미지, 오디오, 비디오)

도구 분포:

- RDBMS: 여전히 60% 투자 (기존 자산)
- 새로운 도구: 40% (벡터 DB, LLM, 특수 엔진)

→ ****Mismatch****! 데이터는 비구조화인데 도구는 구조화 중심

전형적인 데이터 분석 파이프라인의 문제

현재 관행:

[Step 1] 구조화 데이터

관계형 DB	
user_id, age, city	

↓

[Step 2] 비구조화 데이터 추출

사진, 텍스트	
(DB 밖으로 반출)	

↓

[Step 3] 임베딩 생성 (DB 외부)

ML 파이프라인	
BERT, ResNet 실행	
→ 벡터 생성	

↓

[Step 4] 벡터 DB에 로드

Milvus 또는 pgvector	
벡터 저장	

↓

[Step 5] 유사도 검색

별도 쿼리로 검색	
Top-k 결과 수집	

↓

[Step 6] 결과를 다시 원래 DB와 조인

수동 코딩으로	
프로그래밍으로 결합	

문제:

- **복잡함**: 개발자가 6개 단계의 ETL을 수동 관리
- **비효율**: 각 단계마다 데이터를 반복 로드/저장
- **최적화 불가**: DB 옵티마이저가 전체 파이프라인을 볼 수 없음 (일부는 블랙박스)
- **응답 시간**: 각 단계의 레이턴시가 누적되어 전체 응답 시간이 수십 초 이상 가능

5.2 핵심 제안: E-Operator와 E-Scan 프레임워크

E-Operator (Embedding Operator)

임베딩을 관계형 대수의 공식 연산자로 정의:

$$E\mu(R) = \{t \in R, t \rightarrow \mu(t)\}$$

쉽게 말하면:

입력: 관계(테이블) R과 임베딩 모델 μ
 예; $R = (\text{user_id}, \text{user_image})$
 $\mu = \text{ResNet-50}$ 이미지 임베딩 모델

출력: R의 각 튜플(행)에 대해 임베딩 벡터를 추가한 새로운 관계
 예; $(\text{user_id}, \text{user_image}, \text{embedding}[512])$

중요한 성질

1. 역연산 가능성:

$$E\mu^{-1}(E\mu(R)) = R$$

즉, 원래 데이터를 복원할 수 있다.

예시:

임베딩 후 → 벡터 유사도 검색 → 이미지 메타데이터 추출
 ← 원본 user_id , user_image 로 돌아갈 수 있음

2. 관계형 대수와의 호환성:

기존 관계형 규칙이 그대로 적용됨:

$\sigma_p(R)$ = 선택 (조건 p를 만족하는 행만)
 예; $\sigma_{\text{age}>25}(\text{users})$ = 나이 25세 이상 사용자

$\pi_A(R)$ = 프로젝션 (특정 컬럼만 선택)
 예; $\pi_{\text{user_id}, \text{name}}(\text{users})$ = id와 이름만

이런 연산과 $E\mu(R)$ 을 결합할 때 동치 규칙이 성립:

$$\sigma_{\text{age}>25}(E\mu(R)) \equiv E\mu(\sigma_{\text{age}>25}(R))$$

즉, 나이 필터를 임베딩 전에 할지 후에 할지는 동등하다.
 그런데 **비용이 다르다!**

E-Scan 프레임워크

컨텍스트 풍부한 데이터의 소비(consume)·교환(exchange)·캐싱(cache) 인터페이스:

[E-Scan 구현]

- └ Consume: 테이블의 행(row)을 읽을 때
 └ 해당 컨텍스트 데이터도 함께 임베딩
- └ Exchange: 임베딩 결과를 다른 연산자에 전달
 (예; 조인 연산자가 임베딩 벡터를 직접 사용)
- └ Cache: 동일 데이터에 대한 반복 임베딩 방지
 예; 같은 user_image 를 여러 쿼리에서 사용할 때
 첫 번째 쿼리에서 계산한 임베딩을 재사용

5.3 대수적 동치 규칙 (Algebraic Equivalence Rules)

E-Operator가 기존 관계형 대수와 **합성 가능(composable)**함을 보여준다:

선택 푸시다운 (Selection Pushdown)

$$\sigma_p(E\mu(R)) \equiv E\mu(\sigma_p(R))$$

예시:

쿼리: 임베딩한 후, 나이 > 25인 사용자만 필터

[계획 A] 임베딩 먼저 (비효율)	
1. 전체 1,000,000명 사용자	
2. 각각 ResNet 실행 (임베딩)	
→ 1,000,000 × 100ms = 100K초	
3. 나이 필터 적용	
→ 결과 500,000명 (50%)	
비용: 100K초 + IO + 필터링	

[계획 B] 필터 먼저 (효율적)	
1. 나이 > 25 조건으로 필터	
→ 결과 500,000명	
2. 500,000명만 임베딩	
→ 500,000 × 100ms = 50K초	
비용: 필터링 + 50K초 + IO	

결론: 계획 B가 50K초 절감 (50% 성능 향상)

프로젝션 푸시다운 (Projection Pushdown)

$$\pi_A(E\mu(R)) \equiv E\mu(\pi_{\{A_{\text{attr}}\}}(R))$$

즉, 프로젝션할 컬럼을 임베딩 이전에 미리 선택할 수 있다.

예시:

최종 결과에서 user_id와 embedding만 필요하다면,
user_name, user_email은 임베딩 전에 버려도 된다.
(임베딩 모델의 입력에는 user_image만 필요하므로)

5.4 본 논문과의 관계: 왜 정확한 카디널리티가 필요한가

동치 규칙이 있어도 최적의 계획을 선택해야 함

쿼리: 1,000,000명 사용자 중
 - 나이 > 25 (선택도 50%)
 - 임베딩 후 유사 이미지 검색

동치 규칙에 의해 가능한 실행 계획들:

[계획 1] 필터 먼저
 SELECT * FROM users
 WHERE age > 25 -- 50만명 (카디널리티 500K)
 AND embedding_similarity > 0.8

비용 추정:
 - 필터링: $1M \times 1ms = 1,000ms$
 - 임베딩: $500K \times 100ms = 50,000ms$
 - 유사도 검색: $500K \times 10ms = 5,000ms$
 - 합계: 56,000ms

[계획 2] 유사도 검색 먼저
 SELECT * FROM users
 WHERE embedding_similarity > 0.8 -- ???건 (카디널리티 불명)
 AND age > 25

비용 추정:
 - 임베딩: $1M \times 100ms = 100,000ms$ ← 모든 사용자!
 - 유사도 검색: $1M \times 10ms = 10,000ms$
 - 필터링: $??? \times 1ms$ (결과 크기 불명)
 - 합계: $110,000ms + ???$

문제: 유사도 검색의 결과 카디널리티를 모르면?
 → 계획 2의 비용을 정확히 추정할 수 없음
 → 옵티마이저가 계획 1 vs. 2를 비교 불가능

Exqutor의 역할

Exqutor는 정확한 카디널리티를 제공한다:

"유사도 0.8 이상인 이미지는 대략 50,000건입니다"

이제 옵티마이저가:
 - 계획 1: 500K 임베딩 + 500K 검색 = 55,000ms
 - 계획 2: 1M 임베딩 + 1M 검색 + 50K 필터 = 110,050ms
 → 계획 1이 훨씬 나음!

라고 올바르게 판단할 수 있다.

5.5 데이터 특성에 따른 최적 전략의 변화

선택도가 높을 때 (많은 결과)

쿼리: 1,000,000명 중

- 임베딩 유사도 > 0.8 → 500,000건 (50% 선택도)
- 나이 > 25 → 500,000건 (50% 선택도)
- 교집합: 약 250,000건

최적 계획: 필터와 유사도 검색을 병렬로 고려
 SELECT * FROM users
 WHERE age > 25 OR embedding_similarity > 0.8

→ 임베딩과 필터링이 함께 이루어지면 중간 결과 최소화

선택도가 낮을 때 (적은 결과)

쿼리: 1,000,000명 중

- 임베딩 유사도 > 0.9 (매우 유사) → 1,000건 (0.1% 선택도)
- 나이 > 80 (매우 고령) → 100,000건 (10% 선택도)

최적 계획: 더 선택도 높은 것을 먼저 (유사도 검색)
 하지만 그 전에 저비용 필터(나이)를 먼저?

1M 임베딩 비용이 매우 크므로

- 나이 필터 먼저: 100K 사람만 임베딩 → $100K \times 100ms = 10K초$
- 유사도 검색 필터: 결과 약 1K명

vs.

- 1M 모두 임베딩: $1M \times 100ms = 100K초$ (너무 크다)
- 유사도 검색: 1K명

→ 필터 먼저 하는 것이 낫다!

5.6 비전 논문의 한계와 후속 연구의 필요성

이 논문이 제시하는 아이디어는 강력하지만:

1. 구체적 실험 부재
2. 대수적 동치 규칙은 이론적으로 소리가 나지만
3. 실제 구현 시 성능이 얼마나 향상될지는 미지수
4. 비용 모델의 정확성
5. 임베딩 모델의 비용은 입력 크기, 모델 복잡도, 하드웨어에 따라 크게 변함
6. 일반화된 비용 모델을 만들기 어려움
7. 메모리 제약 미고려
8. E-Scan 캐싱은 필요한 메모리가 많을 수 있음
9. 대규모 데이터에서는 메모리 부족 가능
10. 동적 특성 미반영

11. 데이터 분포가 시간에 따라 변함 (concept drift)
12. 최적의 실행 계획도 변할 수 있음

5.7 본 논문과 Exqutor의 관계

이론과 실무의 연결

- [5] 비전 논문 (ICDE 2023)
 - └ E-Operator의 정의 및 동치 규칙 제시
 - └ "임베딩을 관계형 대수에 통합 가능하다"
 - ↓ (후속 연구)
- [6] Context-Enhanced Operators (2023)
 - └ 비전을 실제 구현으로 변환
 - └ 텐서 조인으로 100배 성능 향상
 - └ 인덱스 vs. 스캔 선택도가 중요함을 발견
 - ↓ (파생 연구)
- [Exqutor] (암시적)
 - └ "정확한 카디널리티가 있으면..."
 - └ 옵티마이저가 최적 계획 선택 가능
 - └ 카디널리티 추정을 HNSW 인덱스 탐색으로 해결

Exqutor의 기여

[5]의 비전을 완성하기 위해 필요한 것:

1. **정확한 카디널리티 추정** ← Exqutor가 제공
2. **동적 적응** ← Exqutor의 런타임 샘플링
3. **실제 성능 평가** ← Exqutor의 HNSW, pgvector, VBASE 실험

추가 제기 문제

1. **임베딩 모델의 선택:**
2. 같은 데이터를 여러 임베딩 모델(BERT, RoBERTa, GPT)로 임베딩 가능
3. 각 모델은 비용과 정확도가 다름
4. 옵티마이저가 이를 자동 선택할 수 있을까?
5. **다중 임베딩 필드:**
6. 한 테이블에 여러 임베딩 필드 (텍스트 임베딩, 이미지 임베딩 등)
7. 동적 프로그래밍으로 최적 계획을 찾을 수 있을까?
8. **분산 환경에서의 확장:**
9. 데이터가 여러 노드에 분산될 때
10. 임베딩 계산을 어느 노드에서 할지
11. 네트워크 비용을 어떻게 고려할지
12. **임베딩과 필터 간의 상관관계:**

- 13. 나이와 이미지 유사도가 독립적이지 않을 수 있음
- 14. 고령자의 사진과 젊은이의 사진이 같은 주제일 때 임베딩이 달라질 수 있음
- 15. 이런 상관관계를 비용 모델에 반영?